Three-Week Slide Designer Pipeline Roadmap

Overview

This plan covers three weeks of engineering work to evolve `run_slide_designer.py` from a single-slide prototype into a production-quality pipeline capable of processing entire slide decks in parallel, with significantly improved output quality across all container types. The plan integrates the best patterns from the previous CSS pipeline (`run_css_pipeline.py`) and adds new capabilities: multi-slide deck processing, mimic mode, image passthrough, block diagrams with connectors, text container quality, table accuracy, and a standalone judge module.

> **Note — Week 2 availability**: Rafał will be absent during Week 2. The plan accounts for this: Week 2 work is assigned entirely to Max, Harsh, and Shaz. Tasks that require Rafał (AI agent fine-tuning, prompt engineering, mimic mode) are scheduled in Weeks 1 and 3 only.

Current State Analysis

What Exists in `slide-crafter` (New Pipeline)

The pipeline has a solid foundation:

- **Stage 1 — ContainerRecognizerAgent** (`src/agents/designer/container_recognizer_agent.py`): Visual decomposition via LLM. Identifies containers, decorative layers, upper-layer elements. Quality gate (`cannot_detect` halts pipeline).
- **Stage 1.5 — ContentBinder** (`src/extractors/content_binder.py`): Deterministic centroid-based binding of PPTX shapes to containers. Extracts full table cell data including merge attributes, fill colors, font info.
- **Stage 2 — STUB** (`run_slide_designer.py:261`): All containers marked `RECREATE` . No mimic/copy logic exists.
- **Stage 3+4 — LayoutOrchestrator** (`src/agents/designer/layout_orchestrator.py`): Drives LayoutPlannerAgent → CSSLayoutAgent → Node.js SVG/PPTX. Hardcodes slide dimensions at 13.333×7.5in (`layout_orchestrator.py:419–421`) regardless of actual PPTX dimensions.
- **Stage 3.5 — SVGJudgeAgent** (`src/agents/designer/svg_judge_agent.py`): Optional quality gate. `critique_target="svg_sketcher"` feedback is collected but never forwarded (`layout_orchestrator.py:724`).
- **Node.js pptx_service_node**: Supports only `create_from_layout` . SVG→PPTX converter handles rect/ellipse/text/line/image/group/path. No `<polyline>` , arc paths approximated as straight lines.
- **SlideStructureExtractor** (`src/extractors/slide_structure_extractor.py`): Full PPTX extraction including SmartArt text, per-run paragraph data, table cell colors/merges, theme color resolution.

What Was Better in the CSS Pipeline (`run_css_pipeline.py`)

Key patterns to port:

1. **Parallel TableLayoutAgent** (`css_pipeline.py:1121`): Dedicated table agent runs concurrently with main CSS agent. Uses `__table_placeholder__` tokens in the main layout tree, then injects real table nodes. This produced significantly better table output.
2. **Template context pre-loading** (`css_pipeline.py:301`): Fonts and brand colors extracted from PPTX master before any LLM call. Injected into all agent prompts. Post-processed with `_inject_font_family()` .
3. **Table splitting** (`css_pipeline.py:809`): One PPTX table → multiple logical tables via synthetic `PptxSlideArea` objects with split IDs.
4. **LLM JSON repair** (`layer_skeleton.py:13`): State-machine that handles stray unescaped `"` and `\'` in LLM output before Pydantic validation.
5. **`tableColWidths` adaptive sizing**: `"auto"` , explicit EMU, `"@colN"` reference, `null` — all handled in the CSS agent prompt.

Key Gaps to Fill

Gap	Impact	Owner	Status

Stage 2 is a stub — all containers RECREATE	No mimic mode, no copy mode	Rafał (Week 3)	Pending
No multi-slide deck processing	Can only process one slide at a time	Max	Pending
No image passthrough (source images not carried to output)	Images lost in redesigned slides	Harsh	Pending
No parallel TableLayoutAgent	Table quality poor	Rafał (Week 1, in progress)	In Progress
Hardcoded slide dimensions in orchestrator	Wrong output for non-16:9 slides	Rafał	Done
`judge_feedback_sketcher` never forwarded	Judge feedback loop incomplete	Shaz	Pending
No block diagram / connector support	Block diagrams not renderable	Max	Pending
Text container quality (ornaments, dashed lines, spacing)	Slides look unprofessional	Rafał	Pending
No `_filter_container_layout()` minimal mode key fix	`container_id` vs `id` mismatch	Rafał	Done
No template context pre-loading	Font/color accuracy poor	Rafał	Done
Fix `_extract_structural_constraints()` fragile heuristic	Table detection unreliable	Rafał	Done
Filler container logic	Missing container fill handling	Rafał	Done

Desired End State

After three weeks:

1. **Deck-level CLI**: `run_slide_designer.py --pptx deck.pptx --slides 1,3,5-8` processes selected slides in parallel and assembles a new PPTX with unselected slides preserved in original order.
2. **Mimic mode**: `--mimic` flag causes Stage 2 to assign containers as `COPY` or `COPY_TRANSFORM` instead of `RECREATE`, preserving layout and design from the source slide.
3. **Image passthrough**: Source images embedded in PPTX are extracted, embedded in the SVG as `<image>` elements, and carried through to the output PPTX.
4. **Parallel table processing**: Dedicated `TableLayoutAgent` (ported from CSS pipeline) runs concurrently with the main CSS agent, producing accurate table output.
5. **Block diagram support**: `mixed_elements` containers with connector upper-layer elements rendered correctly via the Node.js `line` element type with arrow markers.
6. **Text container quality**: Ornamental dashed lines, proper content distribution, enumeration formatting, Marvin table text layouts.
7. **Judge module (standalone)**: `SVGJudgeAgent` extracted into an independent module with improved rules, evaluation harness, and configurable thresholds. Can be enabled by default when quality is sufficient.

Verification

```
# Single slide (existing)
python run_slide_designer.py --pptx deck.pptx --slide 3 \
  --design-guidelines guidelines.txt --presentation-style style.txt \
  --pptx-output out.pptx

# Multi-slide deck (new)
python run_slide_designer.py --pptx deck.pptx --slides 1,3,5-8 \
  --design-guidelines guidelines.txt --presentation-style style.txt \
```

```
  --output-dir ./results/

# Mimic mode (new)
python run_slide_designer.py --pptx deck.pptx --slide 2 --mimic \
  --design-guidelines guidelines.txt --presentation-style style.txt \
  --pptx-output out_mimic.pptx
```

What We're NOT Doing

- Full SmartArt/diagram XML reconstruction (only text extraction, visual approximation)
- Connector routing algorithms (connectors drawn as straight lines only)
- Slide master/theme inheritance in output PPTX (output uses minimal theme)
- Automated regression testing suite (manual golden-test comparison only)
- Real-time collaboration or web UI

Team Assignments

Person	Role	Focus Areas
Rafał	Team Lead / Senior Python / AI	Team leadership, quality oversight, AI agent fine-tuning, prompt engineering, Stage 2 (mimic mode, Week 3), parallel TableLayoutAgent port (Week 1, in progress), text container quality, accuracy work with professional slide makers. **Absent Week 2.**
Max	Node.js / PPTX service	pptx_service_node: multi-slide deck utilities, connector/block diagram support, SVG→PPTX improvements, text container Node.js rendering improvements (Phase 2.2), merging branches, end-to-end integration
Harsh	Python / Node (mid-level)	Image passthrough only (Phase 2.1). Accuracy refinement support as capacity allows.
Shaz	AI / Judge module	Standalone judge module only: data preparation (Week 1), prompt improvement + evaluation (Week 2), finalization (Week 3)

Continuous Threads (All Three Weeks)

Two workstreams run in parallel with the weekly phases throughout the entire three weeks:

Accuracy Refinement

Owner: Rafał

Ongoing prompt fine-tuning, rule refinement, and quality review in collaboration with the external professional slide making company. Each week produces a batch of test slides, feedback is collected, and rules are updated. Priority order: space utilization → font hierarchy → table formatting → alignment.

End-to-End Integration and Deck Assembly

Owner: Rafał + Max

The multi-slide deck pipeline is built incrementally. Max builds the PPTX utilities and assembly logic in Week 1; integration is validated each week as new features land; final end-to-end test runs in Week 3.

Week 1: Foundation — Bug Fixes, Table Agent Port, Multi-Slide Skeleton, Text

Quality

Overview

Fix the most impactful bugs, port the parallel table agent from the CSS pipeline, build the multi-slide deck processing skeleton, and begin text container quality work. By end of Week 1, the single-slide pipeline produces noticeably better output (especially tables), and the multi-slide CLI skeleton exists. Rafał is present this week and handles all AI/prompt work.

Phase 1.1: Critical Bug Fixes and Accuracy Baseline

Owner: Rafał | **Status: Partially complete — fixes 1–4 done, fix 5 (text container quality) pending**

> **Meeting note (2026-06-19):** Fixes 1–4 below were completed with AI assistance before the planning meeting. Text container quality (fix 5) remains the active work item for this phase.

1. Fix hardcoded slide dimensions in LayoutOrchestrator

File: `src/agents/designer/layout_orchestrator.py` **Lines**: 419–421 **Problem**: `slide_width_emu = int(13.333 * INCHES_TO_EMU)` ignores the `slide_width_px` / `slide_height_px` inputs that were correctly derived from the PPTX file.

```
# CURRENT (wrong):
slide_width_emu = int(13.333 * INCHES_TO_EMU)
slide_height_emu = int(7.5 * INCHES_TO_EMU)

# FIX: derive from actual slide dimensions passed in
PX_PER_INCH = 128  # same constant used in run_stage15
slide_width_emu = int((inputs.slide_width_px / PX_PER_INCH) * INCHES_TO_EMU)
slide_height_emu = int((inputs.slide_height_px / PX_PER_INCH) * INCHES_TO_EMU)
```

2. Fix `_filter_container_layout()` minimal mode key mismatch

File: `src/agents/designer/content_filters.py` **Line**: ~163 **Problem**: Minimal mode looks for `c.get("container_id", "")` but `Container` serializes as `"id"`.

```
# FIX: check both keys for backward compatibility
entry: Dict = {
    "container_id": c.get("container_id") or c.get("id", ""),
    ...
}
```

3. Port template context pre-loading

File: `src/agents/designer/layout_orchestrator.py` (new method `_preload_template_context()`) **Pattern from**: `css_pipeline.py:301`

Add a pre-stage that calls `PptxAutomizerBridge.extract_template_metadata()` before Stage 1, extracts font names and brand colors from the PPTX master, and injects them into `design_guidelines` and `presentation_style` strings before they reach any LLM. Also add `_inject_font_family()` post-processor on the final layout tree.

4. Fix `_extract_structural_constraints()` fragile heuristic

File: `src/agents/designer/layout_orchestrator.py` **Line**: ~488 **Problem**: `"table" in str(container_data.get("items", [])).lower()` — checks string representation.

```
# FIX: check area_type field directly
area_type = container_data.get("area_type", "") or container_data.get("content_type", "")
if area_type == "table" or container_data.get("table_data"):
    ...
```

5. Text container quality — LayoutPlanner and CSSLayoutAgent prompt refinement *(active)*

Files: `src/agents/designer/layout_planner_agent.py` , `src/agents/designer/css_layout_agent.py`

Add explicit rules for text container archetypes:

- **Enumeration list**: bullet items with consistent indentation, dashed separator lines between items when space allows
- **Marvin table**: matrix layout — rows of text with column alignment, no native table element
- **Decorated text block**: content + ornamental dashes/lines as space fillers when content < 60% of container height
- **Heading + body**: title text + body text with clear visual hierarchy

Add design intent tokens: `enum-list` , `marvin-table` , `decorated-block` , `heading-body` .

Add explicit CSS patterns for each text archetype:

- Dashed separator lines: `elementType="line"` with `style.borderStyle="dashed"` between text items
- Marvin table: nested flex containers (outer row → inner column cells) with `elementType="text"` leaves
- Space distribution: `justifyContent: "space-between"` for enumeration lists

Also refine general accuracy rules:

- Space utilization (Group A in LayoutPlanner)
- Text container fill ratio enforcement
- Font hierarchy rules (Group C)
- Decorative layer placement (Group D)
- Table column width guidance in CSSLayoutAgent

Success Criteria — Phase 1.1:

Automated Verification:

- ☐ Tests pass: `python -m pytest tests/ -v`
- ☐ Type checking: `python -m mypy src/ --ignore-missing-imports`
- ☐ Non-16:9 PPTX (e.g., 4:3) produces correctly sized output

Manual Verification:

- ☐ Run pipeline on 5 test slides; output dimensions match source PPTX
- ☐ Font names from PPTX master appear in output text elements
- ☐ Enumeration list slide: bullets aligned, dashed separators appear between items
- ☐ Decorated text block: ornamental dashes appear when content is sparse

Implementation Note: After completing this phase and all automated verification passes, pause here for manual confirmation from the human that the manual testing was successful before proceeding to the next phase.

Phase 1.2: Parallel TableLayoutAgent Port

Owner: Rafał | **Status: In Progress** (started with AI assistance, pending testing)

> **Meeting note (2026-06-19):** Rafał started this task with AI assistance before the meeting. Originally assigned to Harsh; reassigned to Rafał. Harsh is focused exclusively on image passthrough (Phase 2.1). The implementation is progressing — testing and validation are the next steps.

Port the `TableLayoutAgent` pattern from `css_pipeline/agents/table_layout_agent.py` into the new pipeline.

1. Create `src/agents/designer/table_layout_agent.py`

Port `TableLayoutAgent` from `css_pipeline/agents/table_layout_agent.py` . Key adaptations:

- Input/output models use the new pipeline's `ContainerLayout` / `PptxSlideArea` types
- System prompt retains all table-specific rules: `style.width: "100%"` , `tableData` row×col enforcement, merge continuation cells, adaptive `tableColWidths` , per-side border config

- Output type: `CSSLayoutAgentOutput` (reuse existing model)

2. Modify `CSSLayoutAgent` to emit `__table_placeholder__` nodes

File: `src/agents/designer/css_layout_agent.py`

When `extracted_tables` is non-empty in the input, instruct the agent to emit lightweight placeholder nodes:

```
{
  "id": "C1-table",
  "elementType": "__table_placeholder__",
  "elementConfig": { "tableRef": "C1" },
  "style": { "width": "100%", "height": "40%" }
}
```

3. Add parallel execution in `LayoutOrchestrator`

File: `src/agents/designer/layout_orchestrator.py`

```
from concurrent.futures import ThreadPoolExecutor, as_completed

# In run() per-option loop:
with ThreadPoolExecutor(max_workers=len(extracted_tables) + 1) as executor:
    css_future = executor.submit(self._run_css_agent, ...)
    table_futures = {
        executor.submit(self._run_table_agent, table_id, table_data, ...): table_id
        for table_id, table_data in extracted_tables.items()
    }
    css_output = css_future.result()
    table_outputs = {
        table_id: future.result(timeout=120)
        for future, table_id in ...
    }
```

4. Add `_inject_table_placeholders()` compositor

File: `src/agents/designer/layout_orchestrator.py`

Port `_inject_table_placeholders()` from `css_pipeline.py:1215`. Recursive tree walk replacing `__table_placeholder__` nodes with real table nodes. Copy only positional style keys from placeholder; resolve `flexGrow` -vs- `width` conflict.

Success Criteria — Phase 1.2:

Automated Verification:

- ☐ Tests pass: `python -m pytest tests/ -v`
- ☐ Pipeline runs without error on a slide with a table
- ☐ `TableLayoutAgent` produces valid `CSSLayoutAgentOutput` JSON

Manual Verification:

- ☐ Table in output has correct row count matching source PPTX
- ☐ Table column widths are reasonable (not all equal)
- ☐ Header row is visually distinct from data rows
- ☐ Cell merge spans are preserved

Implementation Note: After completing this phase and all automated verification passes, pause here for manual confirmation from the human that the manual testing was successful before proceeding to the next phase.

Phase 1.3: Multi-Slide Deck Processing Skeleton

Owner: Max

1. Add `--slides` argument to `run_slide_designer.py`

```
parser.add_argument(
    "--slides",
    default=None,
    metavar="RANGE",
    help=(
        "Comma-separated slide numbers or ranges to process "
        "(e.g. '1,3,5-8'). Overrides --slide when provided."
    ),
)
```

Add `parse_slide_selection(slides_arg: str, total_slides: int) -> List[int]` utility function.

2. Add slide extraction and assembly utilities

New file: `src/utils/pptx_deck_utils.py`

```
def extract_slide_to_temp_pptx(source_pptx: Path, slide_number: int, output_dir: Path) -> Path:
    """Extract a single slide into a standalone PPTX for processing."""

def assemble_output_deck(
    source_pptx: Path,
    processed_slides: Dict[int, Path],  # slide_number → processed PPTX path
    output_path: Path,
) -> None:
    """
    Assemble final deck: source slides in original order,
    with processed_slides replacing their originals.
    Uses python-pptx to copy slides.
    """
```

3. Add parallel slide processing loop

File: `run_slide_designer.py`

```
from concurrent.futures import ProcessPoolExecutor, as_completed

def process_single_slide(args: SlideProcessArgs) -> SlideProcessResult:
    """Standalone function for subprocess execution."""
    ...

if selected_slides:
    with ProcessPoolExecutor(max_workers=min(len(selected_slides), 4)) as pool:
        futures = {
            pool.submit(process_single_slide, SlideProcessArgs(slide_n, ...)): slide_n
            for slide_n in selected_slides
        }
        results = {}
        for future in as_completed(futures):
            slide_n = futures[future]
            results[slide_n] = future.result()

    assemble_output_deck(pptx_path, results, output_path)
```

Success Criteria — Phase 1.3:

Automated Verification:

- ☐ `parse_slide_selection("1,3,5-8", 10)` returns `[1, 3, 5, 6, 7, 8]`
- ☐ `assemble_output_deck` produces PPTX with correct slide count
- ☐ Tests pass: `python -m pytest tests/ -v`

Manual Verification:

- ☐ `--slides 1,3` on a 5-slide deck produces a 5-slide output with slides 1 and 3 redesigned
- ☐ Unselected slides are identical to source
- ☐ Slide order is preserved

Implementation Note: After completing this phase and all automated verification passes, pause here for manual confirmation from the human that the manual testing was successful before proceeding to the next phase.

Phase 1.4: Judge Module — Data Preparation (Shaz, Week 1)

Owner: Shaz

Shaz works independently on the judge module throughout all three weeks. Week 1 focus is on understanding the existing judge, extracting it as a standalone module, and preparing the evaluation dataset.

1. Extract `SVGJudgeAgent` into standalone module

Create `judge_module/` as an independent Python package (no dependency on the main pipeline internals):

```
judge_module/
├── __init__.py
├── judge_agent.py          # SVGJudgeAgent (ported from svg_judge_agent.py)
├── judge_rules.py          # Rule definitions (separate from agent)
├── judge_evaluator.py      # Evaluation harness
├── judge_config.py         # Configuration (thresholds, weights)
└── tests/
    ├── test_judge_rules.py
    └── golden_tests/       # PNG pairs: (sketch, source) with expected verdicts
```

2. Prepare golden test dataset

Collect 20+ `(sketch.png, source.png, expected_verdict.json)` triples covering:

- Approved slides (good layout, correct content)
- Rejected slides with critical violations (text overflow, wrong table row count)
- Rejected slides with major violations (poor space utilization, misaligned containers)

3. Build evaluation harness skeleton

File: `judge_module/judge_evaluator.py`

```
def evaluate_judge(golden_dir: Path) -> EvaluationReport:
    """
    Run judge on all (sketch, source, expected) triples in golden_dir.
    Returns precision, recall, accuracy, false positive/negative rates.
    """
```

Success Criteria — Phase 1.4 (Week 1):

Automated Verification:

- ☐ `judge_module` imports without error
- ☐ Evaluation harness runs on at least 5 golden test pairs

Manual Verification:

- ☐ Golden test dataset covers all container types
- ☐ Expected verdicts are documented and justified

Week 2: Quality — Image Passthrough, Text Rendering, Block Diagrams

Overview

Rafał is absent this week. Max, Harsh, and Shaz work independently on their assigned areas. No tasks requiring AI agent prompt engineering are scheduled. The focus is on Node.js-side improvements, image passthrough, and Shaz's judge prompt iteration.

Phase 2.1: Image Passthrough

Owner: Harsh

Source images embedded in PPTX slides must be carried through to the output slide.

1. Python: Extract images from source PPTX

File: `src/extractors/slide_structure_extractor.py` (new method)

```
def extract_slide_images(self, slide_number: int, output_dir: Path) -> Dict[str, Path]:
    """
    Extract all image shapes from a slide to files.
    Returns: {shape_id: image_file_path}
    """
```

Use `python-pptx` `shape.image.blob` and `shape.image.ext` to write image files.

2. Python: Pass image paths through ContentBinder

File: `src/extractors/content_binder.py`

When binding an `area_type="image"` shape to a container, set `source_content["image_path"]` to the extracted image file path. This flows through `content_filters.py` (already preserves `image_path` in minimal mode via `_inject_image_config`).

3. Node.js: Ensure `image-translator.ts` handles file paths correctly

File: `pptx_service_node/ts-src/converters/svg-to-pptx/image-translator.ts`

Verify that `config.imagePath` is correctly embedded in the PPTX output as `<p:pic>` with `<p:blipFill>`. Confirm this works for absolute paths from the Python side.

4. Node.js: Fix `clipPath` loss in SVG→PPTX

File: `pptx_service_node/ts-src/converters/svg-to-pptx/element-dispatcher.ts`

`svg-image.ts` emits `<clipPath>` for rounded corners, but there is no `clipPath` translator. Add handling: when an `<image>` element has a `clip-path` attribute referencing a `<clipPath>` with a `<rect>` child, extract the `rx` / `ry` values and apply them as `<a:prstGeom prst="roundRect">` on the `<p:pic>` shape.

Success Criteria — Phase 2.1:

Automated Verification:

- ☐ `extract_slide_images()` produces image files for all image shapes
- ☐ Tests pass: `python -m pytest tests/ -v`
- ☐ Node.js build succeeds: `npm run build` in `pptx_service_node/`

Manual Verification:

- ☐ Source slide with an image → output PPTX contains the same image
- ☐ Image position and size are approximately correct
- ☐ Rounded-corner images retain their corner radius in output

Implementation Note: After completing this phase and all automated verification passes, pause here for manual confirmation from the human that the manual testing was successful before proceeding to the next phase.

Phase 2.2: Text Container Node.js Rendering Improvements

Owner: Max | **Status: In Progress** (Max has fixes prepared for PR on Monday)

> **Meeting note (2026-06-19):** Reassigned from Harsh to Max. Max confirmed he has bullet and text sizing fixes ready for a PR on Monday. Harsh is focused exclusively on image passthrough.

Implement the Node.js-side rendering improvements needed to support the text container archetypes defined in Phase 1.1.

File: `pptx_service_node/ts-src/renderers/svg-text.ts`

- Ensure `bulletItems` array renders with correct indentation levels
- Ensure `textRuns` with mixed bold/italic/color are rendered correctly
- Add support for `lineHeight` property in text elements

Success Criteria — Phase 2.2:

Automated Verification:

- ☐ Node.js build succeeds: `npm run build`

Manual Verification:

- ☐ Marvin table slide: matrix layout with column alignment
- ☐ Mixed bold/italic text in source is preserved in output
- ☐ Bullet indentation levels are visually correct

Implementation Note: After completing this phase and all automated verification passes, pause here for manual confirmation from the human that the manual testing was successful before proceeding to the next phase.

Phase 2.3: Block Diagrams and Connectors

Owner: Max

Block diagrams (`mixed_elements` containers) with connectors (`upper_layer_elements` of type `connector`) are currently not rendered correctly.

1. Verify connector rendering in Node.js

File: `pptx_service_node/ts-src/renderers/svg-line.ts`

The `line` element type already supports arrow markers (`arrow` , `stealth` , `diamond` , `oval`) and line styles (`solid` , `dash` , `dot`). Verify that:

- `lineStartX/Y` and `lineEndX/Y` are correctly used for connector positioning
- Arrow markers render correctly in SVG and survive SVG→PPTX conversion via `line-translator.ts`
- Elbow connectors (L-shaped) can be approximated with a `path` element

2. Add `polyline` translator to SVG→PPTX

File: `pptx_service_node/ts-src/converters/svg-to-pptx/element-dispatcher.ts`

`<polyline>` is currently not handled. Add a translator that converts `<polyline points="x1,y1 x2,y2 ...">` to a `<p:sp>` with `<a:custGeom>` using `moveTo + lnTo` path commands.

Success Criteria — Phase 2.3:

Automated Verification:

- [ ] Node.js build succeeds: `npm run build`
- [ ] `polyline` SVG element converts to PPTX without error

Manual Verification:

- [ ] Block diagram slide: nodes rendered as rectangles with text
- [ ] Connectors between nodes are visible as lines/arrows
- [ ] Node count in output matches source slide
- [ ] Connection topology is approximately preserved

Implementation Note: After completing this phase and all automated verification passes, pause here for manual confirmation from the human that the manual testing was successful before proceeding to the next phase.

Phase 2.4: Judge Module — Prompt Improvement and Evaluation (Shaz, Week 2)

Owner: Shaz

With the golden dataset from Week 1, iterate on the judge prompts and rules to improve accuracy.

1. Run evaluation baseline

Run the evaluation harness on the full golden test set with the current judge. Record baseline metrics: accuracy, false positive rate, false negative rate.

2. Improve judge rules per container type

File: `judge_module/judge_rules.py`

Separate rule sets for each container type:

- Text containers: fill ratio, bullet alignment, hierarchy
- Tables: row/column count match, header distinction, no cell transposition
- Mixed elements: node count, connector presence
- Images: placeholder presence, approximate position

3. Iterate on judge prompts

File: `judge_module/judge_agent.py`

Refine the system prompt based on evaluation results. Focus on reducing false positives (approving bad slides) first, then false negatives.

4. Re-run evaluation, compare to baseline

Document improvement in accuracy metrics. Target: ≥80% accuracy on golden test set.

Success Criteria — Phase 2.4 (Week 2):

Automated Verification:

- [ ] Evaluation harness runs on full golden test set without error
- [ ] Judge accuracy ≥ 80% on golden test set

Manual Verification:

- [ ] Critique messages are specific and actionable (not generic)
- [ ] Judge correctly handles all container types in the golden set

Week 3: Polish — Mimic Mode, Judge Finalization, Accuracy Refinement, Integration

Overview

Rafał returns this week. Focus is on mimic mode (Stage 2 replacement), block diagram prompt engineering, final accuracy refinement with professional slide makers, judge module finalization and integration, and end-to-end deck pipeline testing.

Phase 3.1: Stage 2 — Mimic Mode (Replace the Stub)

Owner: Rafał

Replace `build_stub_categories()` in `run_slide_designer.py:261` with a real Stage 2 agent.

1. Create `src/agents/designer/container_categorizer_agent.py`

New agent that receives the `ContainerLayout` (with `source_content` populated by Stage 1.5) and the source slide image, and assigns each container one of:

- `RECREATE` — redesign from scratch (default)
- `COPY` — preserve layout and design exactly
- `COPY_TRANSFORM` — preserve layout, allow style improvements

System prompt rules:

- In `--mimic` mode: prefer `COPY` for containers with high confidence (≥0.8) and sufficient content; use `COPY_TRANSFORM` for containers with low confidence or poor visual quality
- In normal mode: all containers → `RECREATE` (same as current stub, but now explicit)
- Special containers (`SC-BOTTOM` , `SC-TOP`) always → `RECREATE`

2. Add `--mimic` flag to CLI

File: `run_slide_designer.py`

```
parser.add_argument(
    "--mimic",
    action="store_true",
    help=(
        "Mimic mode: preserve layout and design from source slide. "
        "Containers are assigned COPY or COPY_TRANSFORM instead of RECREATE. "
        "Useful when the source slide is nearly correct and only needs enhancement."
    ),
)
```

3. Wire mimic mode through the pipeline

Pass `mimic_mode: bool` through `LayoutOrchestratorInput` → `LayoutPlannerAgent` → `CSSLayoutAgent` . When `mimic_mode=True` , the LayoutPlanner is instructed to preserve the source layout structure rather than redesigning it.

Success Criteria — Phase 3.1:

Automated Verification:

- ☐ `ContainerCategorizerAgent` produces valid JSON with all container IDs
- ☐ `--mimic` flag accepted without error
- ☐ Tests pass: `python -m pytest tests/ -v`

Manual Verification:

- ☐ In mimic mode, output slide layout closely matches source slide structure
- ☐ In normal mode, behavior is identical to current stub (all RECREATE)
- ☐ High-confidence containers in mimic mode are preserved faithfully

Implementation Note: After completing this phase and all automated verification passes, pause here for manual confirmation

from the human that the manual testing was successful before proceeding to the next phase.

Phase 3.2: Block Diagram Prompt Engineering

Owner: Rafał

With the Node.js connector work from Phase 2.3 in place, add the Python-side prompt engineering to make block diagrams work end-to-end.

1. Refine ContainerRecognizer for block diagrams

File: `src/agents/designer/container_recognizer_agent.py`

Improve the system prompt rules for `mixed_elements` containers:

- Explicitly identify node shapes (rectangles with text) vs. connector lines
- Assign `upper_layer_elements` of type `connector` with `connected_to` references
- Preserve node count and connection topology in the container description

2. Refine LayoutPlanner for block diagram layout

File: `src/agents/designer/layout_planner_agent.py`

Add block diagram layout patterns:

- `SC-TOP` connector layer: drawn after all content containers, uses absolute positioning
- Node grid: `grid` orientation with equal-sized node containers
- Connection topology: described in `upper_layer_elements` descriptions

Success Criteria — Phase 3.2:

Automated Verification:

- ☐ Tests pass: `python -m pytest tests/ -v`

Manual Verification:

- ☐ Block diagram slide: nodes rendered as rectangles with text
- ☐ Connectors between nodes are visible as lines/arrows
- ☐ Node count and connection topology approximately match source

Implementation Note: After completing this phase and all automated verification passes, pause here for manual confirmation from the human that the manual testing was successful before proceeding to the next phase.

Phase 3.3: Judge Module — Finalization and Integration (Shaz, Week 3)

Owner: Shaz (finalization) + **Rafał** (integration)

Shaz: Finalize judge module

1. **Configurable thresholds** (`judge_module/judge_config.py`):
   - `max_major_violations: int = 2`
   - `min_layout_score: int = 3`
   - `min_content_fidelity_score: int = 3`
   - `enable_by_default: bool = False` (set to `True` when quality is sufficient)
2. **Fix `critique_target="svg_sketcher"` routing**:
   - Currently `judge_feedback_sketcher` is collected but never forwarded (`layout_orchestrator.py:724`)

- Document how this feedback should be routed (forwarded to `CSSLayoutAgent` as additional context)

3. **Final evaluation run**: achieve target accuracy on golden test set, document results.

Rafał: Integrate judge module

Replace the import in `run_slide_designer.py`:

```
# Before:
from src.agents.designer.svg_judge_agent import SVGJudgeAgent, ...

# After:
from judge_module import SVGJudgeAgent, SVGJudgeConfig, SVGJudgeInput, SVGJudgeOutput, SVGJudgePrompts
```

Success Criteria — Phase 3.3:

Automated Verification:

- ☐ `python -m pytest judge_module/tests/ -v` passes
- ☐ Evaluation harness accuracy ≥ 80% on golden test set

Manual Verification:

- ☐ Judge correctly approves a visually good slide
- ☐ Judge correctly rejects a slide with critical violations (text overflow, wrong table row count)
- ☐ Critique messages are actionable and specific
- ☐ Pipeline with `--feedback-loop` runs end-to-end without regressions

Implementation Note: After completing this phase and all automated verification passes, pause here for manual confirmation from the human that the manual testing was successful before proceeding to the next phase.

Phase 3.4: Accuracy Refinement with Professional Slide Makers

Owner: Rafał (primary)

Work with the external professional slide making company to review output quality and refine rules.

1. Collect test cases

Gather 10–15 representative slides covering:

- Title + bullet list
- Two-column layout
- Table (3–8 columns, 4–12 rows)
- Block diagram (4–8 nodes)
- Mixed layout (text + image)
- Decorated text block

2. Run pipeline on all test cases, collect output

```
python run_slide_designer.py --pptx test.pptx --slide N \
  --design-guidelines guidelines.txt --presentation-style style.txt \
  --pptx-output results/slide_N.pptx --feedback-loop
```

3. Review with professional slide makers and implement feedback

Priority order based on frequency of violations:

1. Space utilization
2. Font hierarchy

3. Table formatting
4. Alignment and grid adherence

Success Criteria — Phase 3.4:

Automated Verification:

- ☐ Tests pass: `python -m pytest tests/ -v`

Manual Verification:

- ☐ Professional slide maker rates ≥7/10 slides as “acceptable” or better
- ☐ No slide has critical space utilization violations (empty regions >15%)
- ☐ Table slides have correct row/column counts in all test cases
- ☐ Font hierarchy is consistent across all test cases

Implementation Note: After completing this phase and all automated verification passes, pause here for manual confirmation from the human that the manual testing was successful before proceeding to the next phase.

Phase 3.5: End-to-End Integration and Deck Assembly

Owner: Rafał + Max

1. Max: Merge branches

Review and merge:

- `table_layout_agent.py` branch (Phase 1.2, Rafał)
- Image passthrough (Phase 2.1, Harsh)
- Text rendering improvements (Phase 2.2, Max)
- Any other Node.js fixes

2. Rafał + Max: End-to-end deck pipeline test

Run the full deck pipeline on a 10-slide presentation:

```
python run_slide_designer.py --pptx full_deck.pptx --slides 1-10 \
  --design-guidelines guidelines.txt --presentation-style style.txt \
  --output-dir ./results/
```

Verify:

- All 10 slides processed without error
- Output deck has 10 slides in correct order
- Unselected slides (if any) are identical to source
- Processing time is acceptable (< 5 minutes for 10 slides with parallelism)

3. Rafał: Final CLI polish

- Add `--output-dir` argument for multi-slide output
- Add progress reporting (slide N of M)
- Add `--dry-run` flag that runs Stage 1 only for all slides (quick quality check)
- Ensure `--mimic` works correctly in multi-slide mode

Success Criteria — Phase 3.5:

Automated Verification:

- ☐ Tests pass: `python -m pytest tests/ -v`

- [ ] Type checking: `python -m mypy src/ --ignore-missing-imports`
- [ ] Node.js build succeeds: `npm run build`

Manual Verification:

- [ ] Full 10-slide deck processed successfully end-to-end
- [ ] Output deck opens in PowerPoint without errors
- [ ] Slide order is correct
- [ ] Processing time < 5 minutes for 10 slides
- [ ] `--mimic` mode on a 5-slide deck produces output that closely matches source layouts

Implementation Note: After completing this phase and all automated verification passes, pause here for manual confirmation from the human that the manual testing was successful before proceeding to the next phase.

Optional: Chart Support (Backup Task)

> **This is an optional stretch goal.** It is not scheduled in any week. If a team member finishes their assigned work early — particularly during Week 2 when Rafał is absent — this is a well-defined task that can be picked up independently. There are existing proof-of-concept implementations for charts that may be usable; check with Rafał before starting.

Chart Support — Max or Harsh (if capacity allows)

There are existing proof-of-concept chart implementations that may already work well. The task here is to wire chart data extraction through the Python pipeline so that chart data reaches the Node.js renderer.

1. Python: Extract chart data from PPTX

File: `src/extractors/slide_structure_extractor.py` (new method `_extract_chart_data()`)

Use `python-pptx` to extract chart data from `MSO_SHAPE_TYPE.CHART` shapes:

- Chart type (bar, column, line, pie)
- Series names and data values
- Category labels
- Chart title

Store as `chart_data` dynamic attribute on `PptxSlideArea` (same pattern as `table_data`).

2. Python: Pass chart data through ContentBinder

File: `src/extractors/content_binder.py`

When binding a `content_type="chart"` container, set `source_content["chart_data"]` from the extracted chart data. The `_inject_chart_config()` in `layout_content_injector.py` already handles `chart_data` → `chartData` mapping.

3. Node.js: Verify chart rendering in `svg-chart.ts`

File: `pptx_service_node/ts-src/renderers/svg-chart.ts`

Verify that the existing chart renderer handles bar/column/line/pie charts correctly with the extracted data. Fix any rendering issues found during testing.

Success Criteria (if picked up):

Automated Verification:

- [ ] `_extract_chart_data()` returns valid chart data for a PPTX with charts
- [ ] Node.js build succeeds: `npm run build`

Manual Verification:

- ☐ Bar chart in source PPTX → bar chart in output with correct series count
- ☐ Chart title is visible in output

Testing Strategy

Unit Tests

- `parse_slide_selection()` : edge cases (single slide, range, mixed, out-of-bounds)
- `assemble_output_deck()` : slide count, order, content preservation
- `_filter_container_layout()` minimal mode: `container_id` vs `id` key fix
- `_extract_structural_constraints()` : table detection with `area_type` field
- `TableLayoutAgent` : valid output for 3×3, 5×8, merged-cell tables

Integration Tests

- Single slide pipeline: PPTX → PNG → Stage 1 → Stage 1.5 → Stage 2 → Stage 3+4 → PPTX
- Multi-slide pipeline: 3-slide deck, process slides 1 and 3, verify output
- Mimic mode: high-confidence containers preserved, low-confidence containers redesigned
- Image passthrough: source image appears in output PPTX

Manual Testing Steps

1. Run on 5 representative slides (title+bullets, table, block diagram, mixed)
2. Open output PPTX in PowerPoint/LibreOffice, verify no rendering errors
3. Compare output to source: layout structure, content completeness, visual quality
4. Run with `--feedback-loop` enabled, verify judge verdicts are reasonable
5. Run `--mimic` mode, verify source layout is preserved
6. Run multi-slide mode on a 5-slide deck, verify assembly

Performance Considerations

- **Parallel slide processing**: Use `ProcessPoolExecutor` (not `ThreadPoolExecutor`) for slide-level parallelism to avoid Python GIL contention with LLM calls. Max 4 workers to avoid AWS Bedrock rate limits.
- **Parallel table processing**: Use `ThreadPoolExecutor` within a single slide (I/O-bound LLM calls). Max workers = number of tables + 1.
- **Node.js subprocess**: Each slide spawns its own `PptxAutomizerBridge` context. For multi-slide processing, consider a shared bridge pool to avoid subprocess startup overhead.
- **Image extraction**: Extract images once per slide, cache in temp directory. Clean up after assembly.

Migration Notes

- The `build_stub_categories()` function in `run_slide_designer.py:261` is replaced by `ContainerCategorizerAgent` in Phase 3.1. The stub remains as a fallback when `--mimic` is not specified and the new agent fails.
- The `judge_feedback_sketcher` variable in `layout_orchestrator.py:724` is currently unused. Phase 3.3 documents the intended routing; actual forwarding to `CSSLayoutAgent` is implemented as part of the judge module integration.
- The `table_data` dynamic attribute pattern (`pptx_area.table_data = table_data`) is preserved for backward compatibility. A future cleanup task should make it a declared field on `PptxSlideArea` .

Weekly Milestones Summary

Week	Milestone	Owners
Week 1	Critical bugs fixed , text container quality prompts (in progress), parallel table agent (Rafał, in progress), multi-slide CLI skeleton, judge module extracted	Rafał, Max, Shaz
Week 2 *(Rafał absent)*	Image passthrough (Harsh), Node.js text rendering (Max, PR Monday), block diagram connectors (Max), judge prompt iteration (Shaz)	Max, Harsh, Shaz
Week 3	Mimic mode, block diagram prompts, judge finalization + integration, accuracy refinement, end-to-end deck test	Rafał, Max, Harsh, Shaz

References

- New pipeline entry point: `run_slide_designer.py`
- Previous CSS pipeline: `/home/rafal/work/pptx_processing_pipeline/experiments/css_pipeline/run_css_pipeline.py`
- Stage 2 stub: `run_slide_designer.py:261`
- Hardcoded dimensions bug: `src/agents/designer/layout_orchestrator.py:419–421`
- Key filter bug: `src/agents/designer/content_filters.py:163`
- Parallel table agent (source): `/home/rafal/work/pptx_processing_pipeline/experiments/css_pipeline/agents/table_layout_agent.py`
- Template context pre-loading (source): `/home/rafal/work/pptx_processing_pipeline/experiments/css_pipeline/css_pipeline.py:301`
- Node.js SVG→PPTX converter: `pptx_service_node/ts-src/converters/svg-to-pptx/`
- Node.js SVG renderer: `pptx_service_node/ts-src/renderers/`